

SOFTWARE

1. Introduction

The automated vertical blinds project aims to provide smart control of window blinds using a **NodeMCU (ESP8266)**, a **DC motor with L293D driver**, and a mobile interface developed in **MIT App Inventor**. The system allows the user to:

- Open, close, or stop the blinds via manual buttons,
- Control the blinds using **voice commands**,
- Prevent damage using **limit switches** to stop the motor at the end positions.

This report details the programming aspects of both the **embedded firmware** and the **mobile interface**, demonstrating how they integrate to form a user-friendly, efficient home automation solution.

Code Breakdown: Arduino (ESP8266) Side

1. Libraries and Global Setup

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
```

- These libraries allow the NodeMCU to create a Wi-Fi access point and run an HTTP server.

```
const char* ssid = "Blinds_Control";
const char* password = "12345678";
```

- These define the SSID and password for the Wi-Fi access point created by the NodeMCU.

```
ESP8266WebServer server(80);
```

- Initializes a web server on port 80.

2. Pin Definitions and Variables

```
#define IN1 D1
#define IN2 D2
#define ENA D3
#define LIMIT_UP D5
#define LIMIT_DOWN D6
```

- These define the motor control pins and limit switch inputs.

```
int motorSpeed = 200;
```

- Motor speed (PWM value out of 1023 for ESP8266).

```
enum MotorState { STOPPED, MOVING_UP,
MOVING_DOWN };
MotorState motorState = STOPPED;
```

- Defines a custom type to track motor status.

3. Debounce Function for Limit Switches

```
bool isLimitPressed(int pin) {
    int count = 0;
    for (int i = 0; i < 5; i++) {
        if (digitalRead(pin) == LOW) count++;
        delay(2); // Total debounce time =
10ms
    }
    return (count >= 4);
}
```

```
}
```

- Checks the limit switch status with debounce logic to avoid false triggering.

4. Motor Control Functions

```
void stopMotor()
```

- Stops the motor completely.

```
void moveUp()
```

- Sets motor to move in the "up" direction.

```
void moveDown()
```

- Sets motor to move in the "down" direction.

Each function adjusts the motor driver pins accordingly.

5. Web Request Handlers

```
void reqMoveUp()
```

```
void reqMoveDown()
```

```
void reqStop()
```

- These handle incoming HTTP requests to `/open`, `/close`, and `/stop`, respectively, and respond with plain text status updates.

```
void handleNotFound()
```

- Responds with a 404 message for any undefined routes.
-

6. Setup Function

```
void setup() {  
    ...  
    WiFi.softAP(ssid, password);  
    ...  
    server.on("/open", reqMoveUp);  
    server.on("/close", reqMoveDown);  
    server.on("/stop", reqStop);  
    server.onNotFound(handleNotFound);  
    server.begin();  
}
```

- Initializes GPIO pins, serial communication, Wi-Fi AP, and starts the HTTP server with the necessary routes.

7. Loop Function

```
void loop() {  
    server.handleClient();  
  
    if (millis() - lastCheck >= 50) {  
        ...  
        if (motorState == MOVING_UP &&  
isLimitPressed(LIMIT_UP)) stopMotor();  
        if (motorState == MOVING_DOWN &&  
isLimitPressed(LIMIT_DOWN)) stopMotor();  
    }  
}
```

- Continuously checks for incoming client requests.
- Every 50ms, it checks if a limit switch has been triggered and stops the motor if necessary.

3. MIT App Inventor Programming Breakdown

The MIT App Inventor interface allows manual and voice control through a visual block-based system.

3.1 Manual Button Controls

Each button (Open, Close, Stop) sends an HTTP GET request to the corresponding route on the NodeMCU and updates a status label:

Example (Open Button):

```
when Button_Open.Click  
→ set Web1.Url to  
"http://192.168.4.1/open"  
→ call Web1.Get  
→ set Label_Status.Text to "Status:  
Opening Blinds..."
```

- **Close** and **Stop** buttons work similarly by calling /close and /stop.

3.2 Voice Control with SpeechRecognizer

The app uses the **SpeechRecognizer** component to allow voice commands.

Process:

1. User taps mic button (Button_1.Click)
2. SpeechRecognizer.GetText is triggered
3. After recognizing speech:
 - If the text contains "open" → send /open request
 - If it contains "close" → send /close request

- Else → send /stop as fallback

Example:

```
if result contains "open" → /open  
else if result contains "close" → /close  
else → /stop
```

3.3 Status Display

After each command, the app updates a status label to show the user what action was taken (e.g., "Status: Opening Blinds...").

4. System Integration Flow

1. **User connects** phone to the ESP8266's hotspot (Blinds_Control).
2. **App sends** HTTP GET requests to control the blinds.
3. **NodeMCU reads** requests and activates motor accordingly.
4. **Limit switches** stop the motor when full open/close is reached.
5. **Status feedback** is shown on the app screen.

5. Conclusion

The programming of this automated vertical blinds system successfully integrates:

- Low-level motor control with safety logic (limit switches),
- Wi-Fi-based web server for wireless control,
- A mobile app that enables **both button and voice control** using a user-friendly GUI.

